Week 4- What more to check in EDA (Pitfalls)?

Dr. Smitha K G
Senior Lecturer,
College of Computing and Data Science,
Nanyang technological University

NANYANG
TECHNOLOGICAL
UNIVERSITY
SINGAPORE

Target Leakage & Target Validity

Need to check

- Any feature derived *directly or indirectly* from the target
- Features using future information (post outcome feature)

Why critical?

- Produces unrealistically high accuracy (overfitting in training data)
- Fails catastrophically in deployment

Classic examples

- FinalGrade used to predict Pass/Fail

```
Data columns (total 4 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   Income           2000 non-null   float64
 1   CreditScore      2000 non-null   float64
 2   AmountRecovered  2000 non-null   float64
 3   Default          2000 non-null   int64
dtypes: float64(3), int64(1)
```

Target

Correlated Feature	Post-Outcome Feature
income	Amount_recovered
creditscore	

Amount recovered is a post-outcome feature because its value is determined only after the borrower has either defaulted or repaid, making it causally dependent on the target variable and unavailable at prediction time

Data Representativeness & Sampling Bias

Need to check

- Who/what is missing from the dataset?
- How was the data collected and does the dataset represent the real population?

Why critical?

- Models generalize only to the population they see
- Leads to systemic bias and unfair predictions and this Invalidates evaluation metrics

Examples

- Loan datasets with only approved applicants
- Medical datasets from a single hospital
- Student datasets from only high-performing institutions

Class balance ≠ population balance.

Temporal Leakage / Ignoring Time- introduces concept drift

Concept drift occurs when the statistical relationship between inputs (X) and target (y) changes over time.

A model that ignores time assumes the world never changes.

Need to check

- Time dependency in features or labels
- Whether feature–target relationships change over time
- Whether train/test splits respect time

Core Strategies to Handle Drift

1.Detect drift

2.Adapt the model

3.Choose drift-robust algorithms

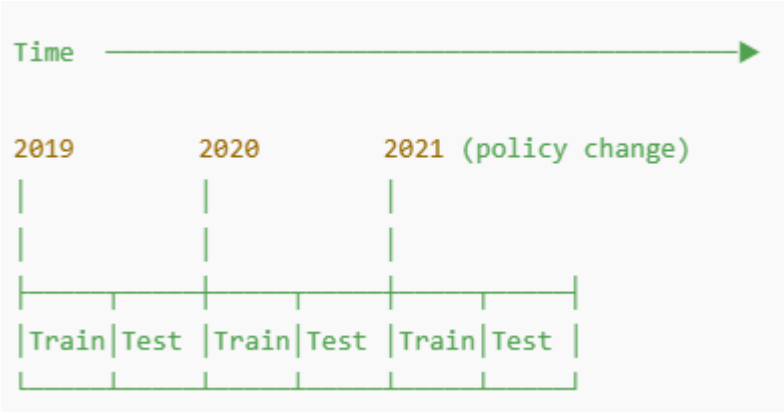
Why critical?

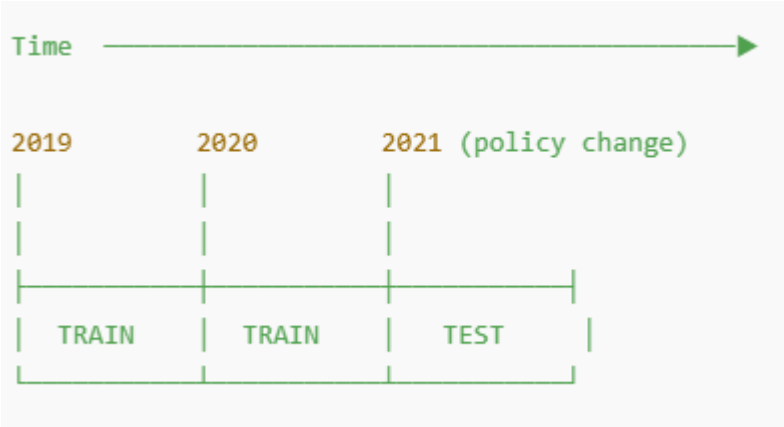
- Random splits give false confidence
- Models degrade rapidly after deployment and Decisions become biased or unsafe

Temporal Leakage / Ignoring Time- introduces concept drift- example

Student performance across yeas (concept drift + temporal leakage)

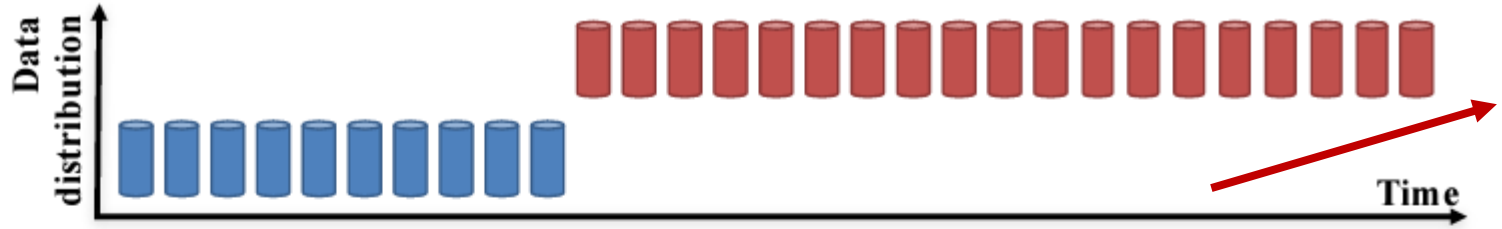
- You generate data for 3 years (2019–2021).
- Students' StudyHours are similar across years.
- But the grading rule changes in 2021 (becomes stricter), so the same study hours lead to more failures in 2021.
- Then you do a WRONG random train/test split, which mixes years.
- Because the model sees some 2021 patterns during training, the test accuracy looks better than what you'd get in real deployment, where you train on past years and predict future years.





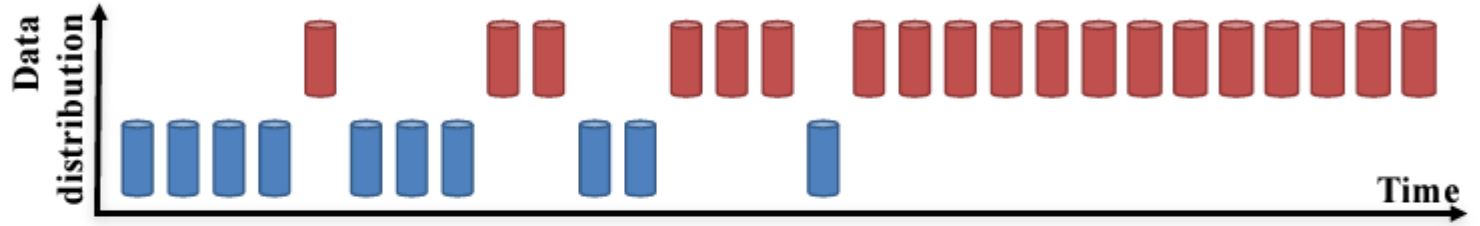
Drift is inevitable; failure comes from treating all drift the same.

Sudden Drift:



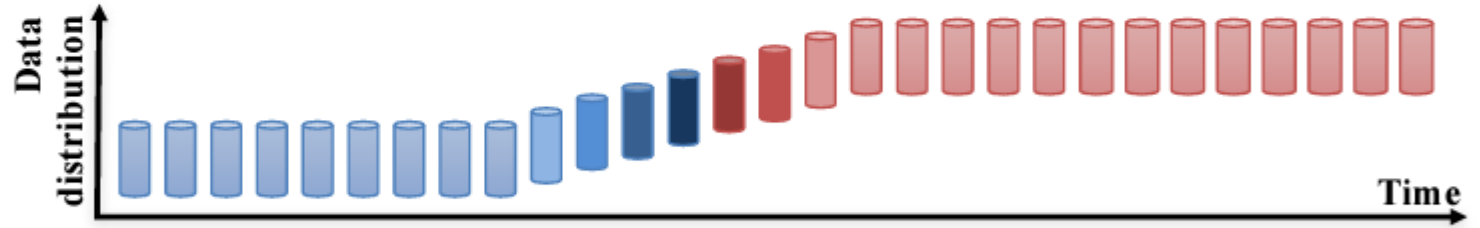
A new concept occurs within a short time.

Gradual Drift:



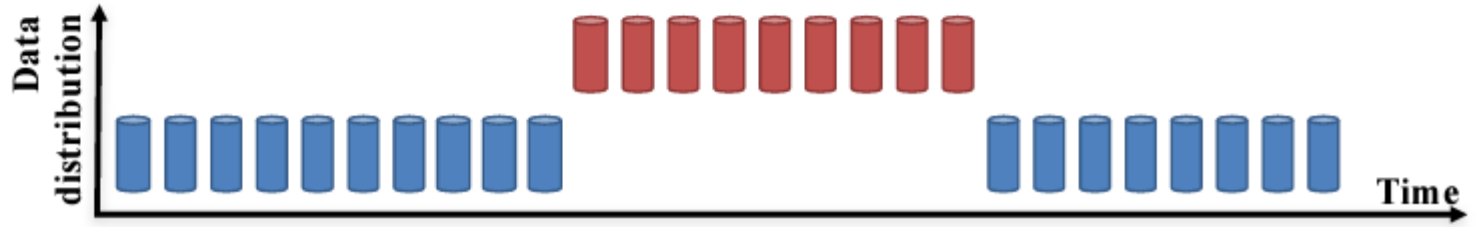
A new concept gradually replaces an old one over a period of time.

Incremental Drift:



An old concept incrementally changes to a new concept over a period of time.

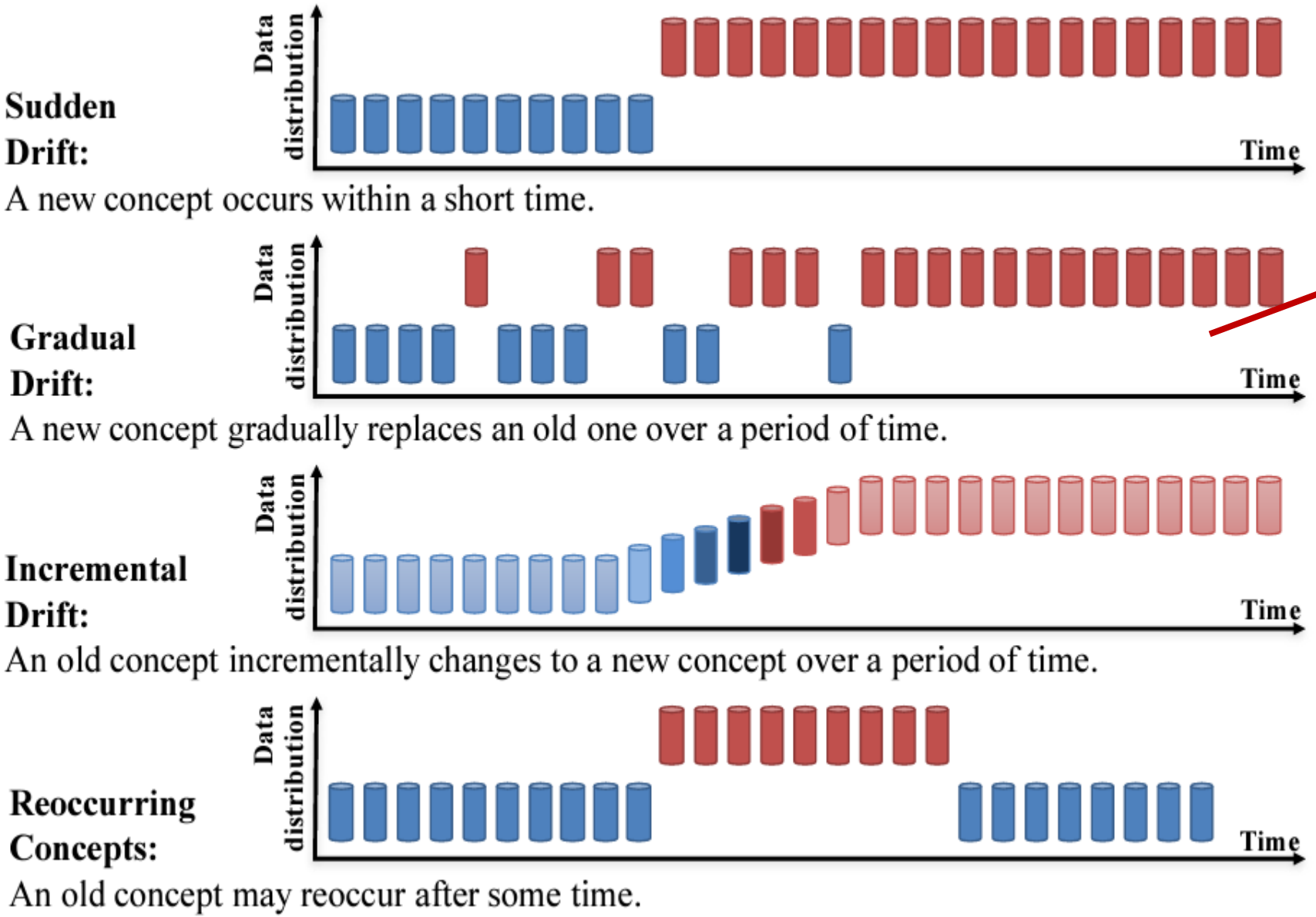
Reoccurring Concepts:



An old concept may reoccur after some time.

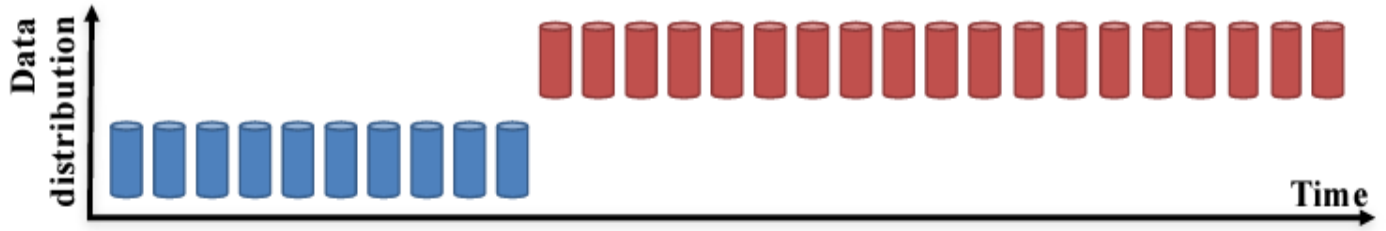
- Old concept becomes invalid almost instantly
- Past data is misleading
- Train only on *post-drift* data
- ML algo like Random forest with a sliding window, Logistic regression with weighted samples or gradient boosting (windowed)
- Ex:New grading policy

Drift is inevitable; failure comes from treating all drift the same.

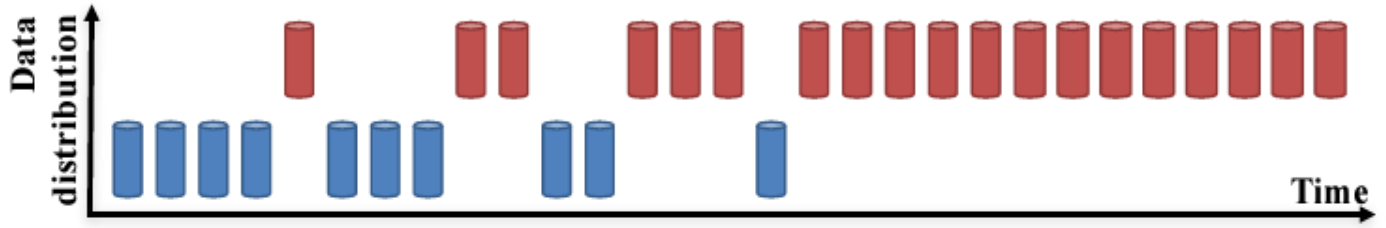


- Old and new concepts coexist
- Proportions shift over time
- Controlled forgetting
- Sliding / rolling window training (Train on last *N* months)
- Time-decayed weighting (Recent samples weighted more)
- Ex: Market behavior changes
- ML algorithms like Random forest with a sliding window, Logistic regression with weighted samples or gradient boosting (windowed), Kalman filter (time series)

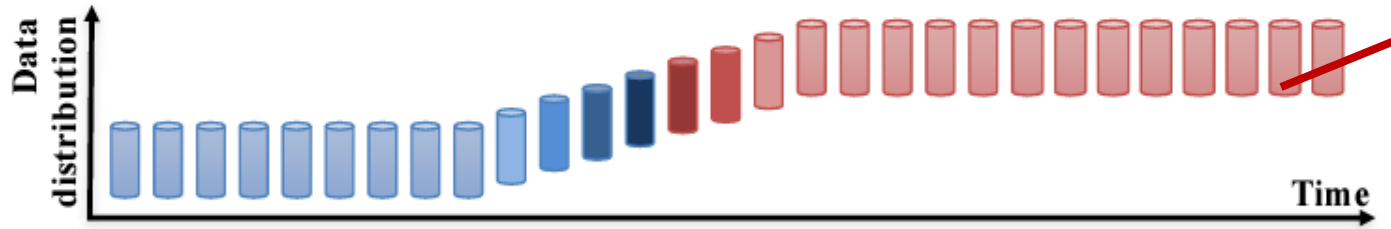
Drift is inevitable; failure comes from treating all drift the same.



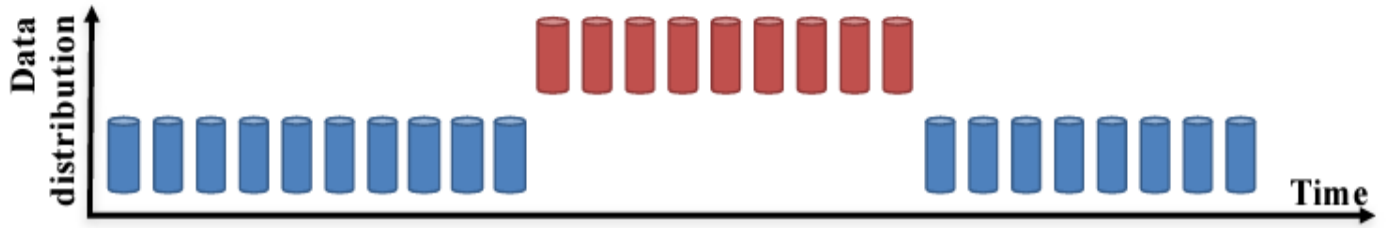
Sudden Drift:
A new concept occurs within a short time.



Gradual Drift:
A new concept gradually replaces an old one over a period of time.



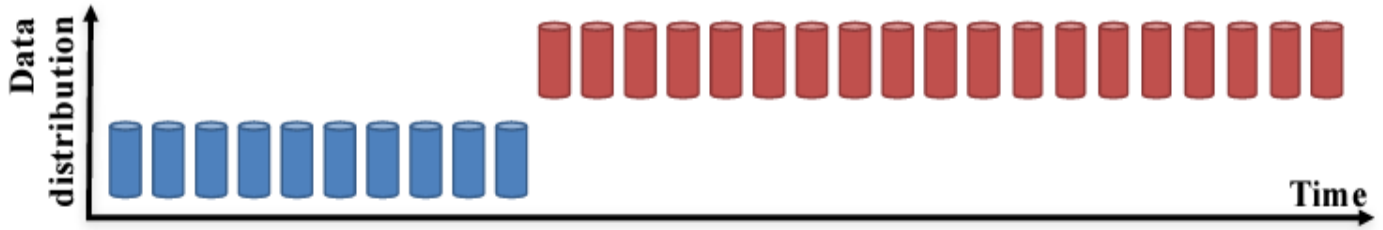
Incremental Drift:
An old concept incrementally changes to a new concept over a period of time.



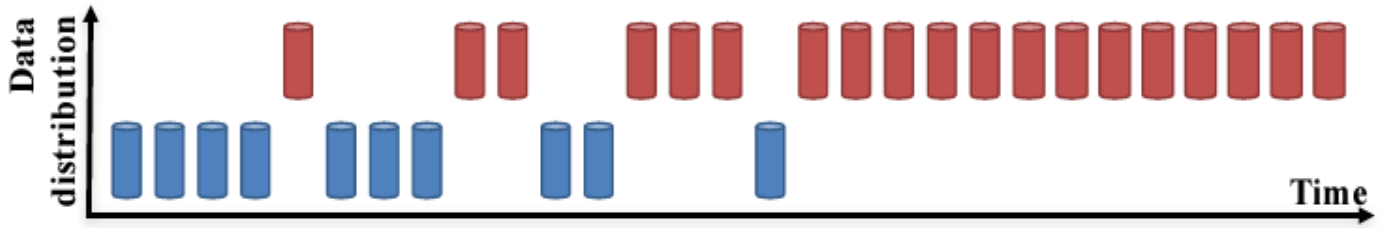
Reoccurring Concepts:
An old concept may reoccur after some time.

- Incremental Drift (Continuous shift)
- Concept changes smoothly
- No clear “before vs after”
- Incremental model updates
- Adaptive learning rates
- Ex:Inflation effects, Skills acquired over time

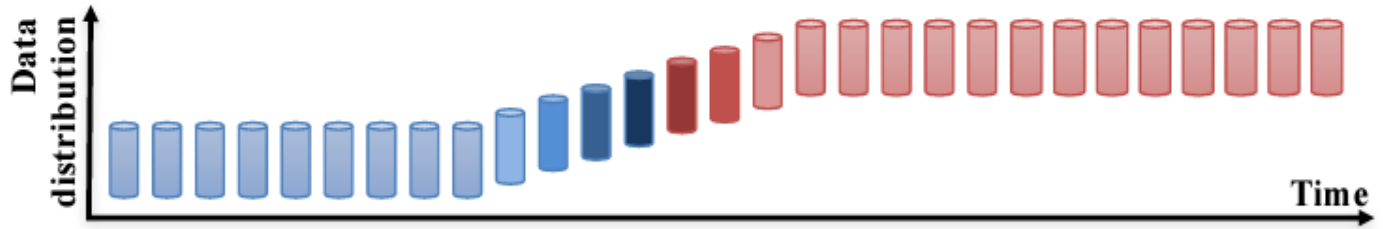
Drift is inevitable; failure comes from treating all drift the same.



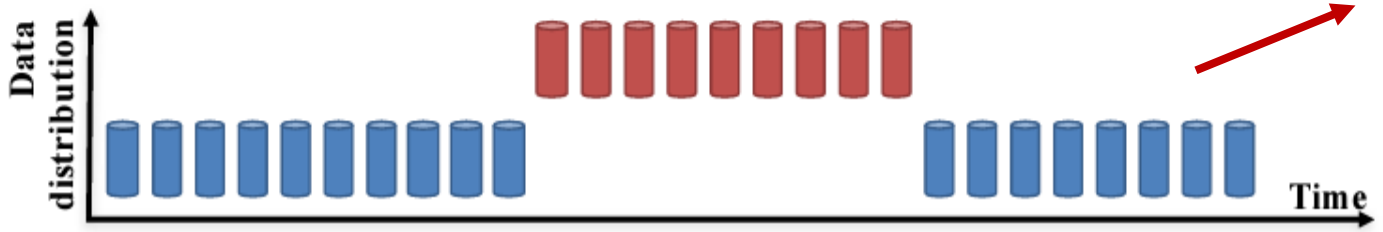
Sudden Drift:
A new concept occurs within a short time.



Gradual Drift:
A new concept gradually replaces an old one over a period of time.



Incremental Drift:
An old concept incrementally changes to a new concept over a period of time.



Reoccurring Concepts:
An old concept may reoccur after some time.

- Recurring Concepts (Cyclic / seasonal) Old concepts reappear
- Forgetting past models is harmful
- Concept changes smoothly
- Context-aware model selection and meta leaning (learn when to switch models)
- Ensemble with regime detection
- Ex:Pandemic vs non-pandemic behavior

Multicollinearity

What to check

- Strong correlations among predictors
- Variance Inflation Factor (VIF > 5–10)
- Redundant engineered features
- Semantically overlapping variables

Why it is critical

- Feature importance becomes unreliable
- Causal and policy interpretations become invalid

Multicollinearity does not kill accuracy — it kills explanations

Examples

- Predict house price using: ‘HouseAge’ ‘YearsSinceRemodel’ as these features are strongly correlated.

House age, YearsinceRemodel example

Model tries: (linear regression)
Price = a × HouseAge + b × YearsSinceRemodel

But both ≈ Age

HouseAge ≈ -4065 ← absorbs almost all effect
YearsSinceRemodel ≈ +80 ← leftover noise

High Variance Inflation Factor(VIF) means the model cannot decide who deserves credit, even though it can still predict well.

Multicollinearity does NOT break predictions.
It breaks explanations.

	HouseAge	YearsSinceRemodel	Price
0	39	39	154235.215225
1	29	12	173063.710539
2	15	15	243721.313911
3	43	43	132788.721639
4	8	8	267758.230229
5	21	20	210289.952804
6	39	39	141366.780974
7	19	19	219967.104980
8	23	23	232020.262818
9	11	11	255773.969106

Model is asking this big question?
What extra information does YearsSinceRemodel add once HouseAge is known?

Effects of Multicollinearity

When two predictors are **highly correlated**, the regression model:

- Cannot uniquely decide **which feature deserves credit**
- Small noise → large coefficient changes
- Coefficients may flip sign even if the true relationship is known
- Interpretation contradicts domain logic

- The model is still minimizing error correctly — but explanations become unstable.
- It is good to : Drop redundant features and combine semantically similar features

Why EffectiveAge is BETTER than PCA here?

PCA	EffectiveAge
Removes correlation	Removes redundancy
Loses meaning	Preserves meaning
Hard to explain	Easy to explain
Model-centric	Domain-centric

Prefer domain-informed feature consolidation over blind dimensionality reduction when interpretability matters

Label Noise/Quality

What to check

- Incorrect, inconsistent (same input gets different labels depending on annotator, time and context) or proxy labels
- Eg: Self reported survey labels- On a scale of 1–10, how stressed are you?

Why it is critical

- Makes decision boundary fuzzy
- Regularization cannot fix wrong labels
- Model confidence is reduced as it fails real learning objectives

Examples

- Human-annotated sentiment analysis, Eg:-Classifying reviews as Positive / Neutral / Negative (“The phone is amazing, but the battery dies in 3 hours.”)
- One can say it to be Positive (phone amazing), another neutral (one positive one negative), another negative(as it dies in 3 hours) – Subjective interpretation and even fatigue

Outlier semantics

What is it

- Outlier semantics means understanding what an outlier represents in the real world, not just whether it is statistically extreme.
- An outlier is not automatically a noise or error— it may be a valid, rare, or important population or a distinct subgroup.

Why it is critical

- Model learns only “average” or “ dominant” behavior once outlier is removed.
- Sometimes outliers are our major signals to learn (understanding a rare medical condition)

Examples

- Rare medical condition- Genetic disorder for 1% of population
- As they are statistically and outlier, the model may remove that and never learns the disease patterns.

TIER 3 — Model compatibility and Optimization

(Model works but not optimally- performance degraded)

Failure	Looks Good Because	Fails Because
Feature Scaling and Distribution shape (skew)	1. Model trains without errors 2. Metrics may look acceptable on small datasets	1. Distance-based models distort similarity due to scale imbalance 2. Heavy-tailed/skewed features dominate gradients
Dimensionality and feature redundancy	Adding features initially improves training performance	1. Training time, memory usage, and inference cost increase unnecessarily 2. Redundant features inflate variance and overfitting risk

Tier-3 risks are often mistaken for “model weakness” when the root cause lies in preprocessing and representation.